

Stat 414 Notes

Table of contents

Course Goals for STAT 414	3
1 Introduction to Sampling	4
1.1 Introduction	4
1.2 Considerations in planning and executing a survey	8
2 An Introduction to Probability Samples	16
2.1 Probability Sampling Definitions and Notation	17
2.2 Probability Samples: Inference Basics	19
3 Simple Random Samples	22
3.1 SRS: Selecting the Sample	23
3.2 Estimation from a SRS	29
3.2.1 Estimating a population mean	29
3.2.2 Estimating a population total	34
3.2.3 Estimating a population proportion	35
3.3 Sampling Weights	38
3.4 Confidence Intervals	42
3.5 Sample Size	48
1. Decide how much precision you need	48
2. Find an equation involving n and the precision	48
3. Estimate anything you need to and solve for n	51
4. Go back to square one when the n turns out to be enormous	51
3.6 Systematic Sampling	53
3.7 Considerations for Choosing an SRS Design	55
When Shouldn't You Use an SRS?	55
Why Should You Use an SRS?	55

Course Goals for STAT 414

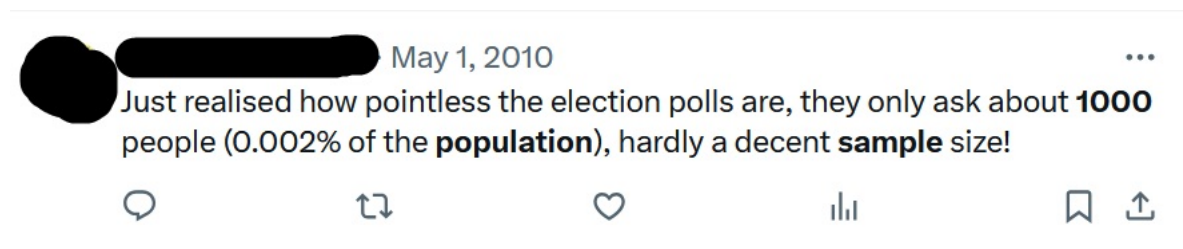
STAT 414 is an introduction to sampling. By the end of the course, you will:

- Learn the basic methodology of how classical survey samples are conducted based on simple random sampling.
- Recognize sampling and non-sampling errors.
- Be able to use extra information in sampling via ratio estimators and regression estimators.
- Be able to define and use stratified samples and know why they are often better than SRS.
- Be able to design one and two stage cluster samples and know their key properties.
- Recognize strengths and weaknesses of different sampling approaches.
- Use standard R packages to analyze survey data and estimate population means, proportions, totals, and more.

1 Introduction to Sampling

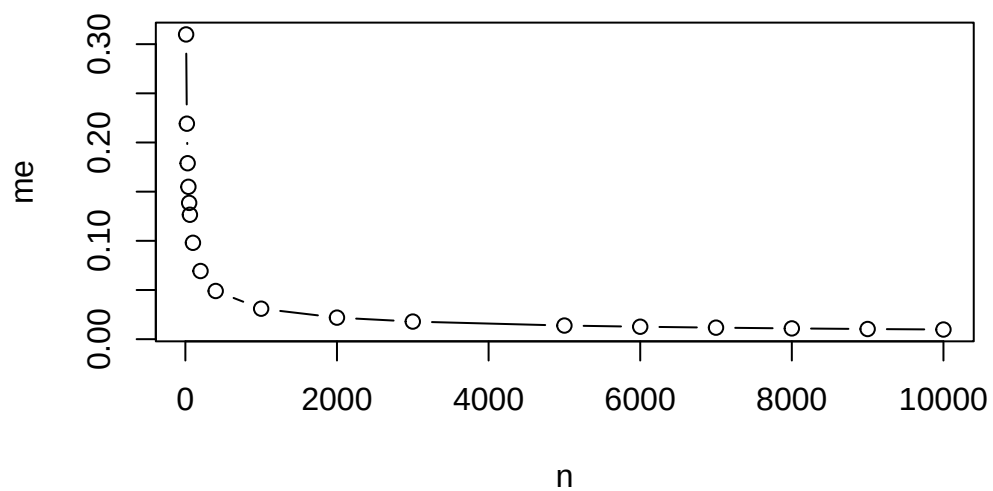
1.1 Introduction

If you spend any time at all on social media (especially in an election year), you're likely to come across a post like this:



What do you think? Do you agree?

```
n<-c(10,20,30,40,50,60,100,200,400,1000,2000,3000,5000,6000,7000,8000,9000,10000)
me<-1.96*sqrt(0.25/n)
plot(n,me,type="b")
```



So why even sample?

1. Cheaper
2. Faster
3. **More Accurate** (?!?!?)

  Aug 26, 2020 ...

Wednesday open question: What's one thing that's very hard for most people to believe, but which is actually true?

 414  101  241   

  ...

a surveyed sample of 1,000 individuals, if representative of the larger population, is sufficient to make accurate predictions about a population of 300 million people

3:18 PM · Aug 26, 2020

What's key here?

What we will spend this semester studying is how to ensure this, to make sure we can make accurate predictions. This will involve answering questions focused on:

- Survey design
- Preparation
- Collect and prepare data
- Data analysis

How is sampling different from previous courses, like STAT 212?

How is sampling different from previous courses, like STAT 262 or STAT 380?

How is sampling different from previous courses, like STAT 102 or STAT 318?

However, like previous courses, sample surveys do have an overarching process, some of which overlap with steps/processes we've seen before. We'll go through the principal considerations in a sample survey, along with some definitions and (possibly) new notation.

1.2 Considerations in planning and executing a survey

1. Objectives of the survey
2. **Defining the population** In sampling, the word **population** needs some clarification. Generally, it means the larger set from which the sample is chosen, and it's the larger group we wish to make inference/draw conclusions about. And a **sample** is a subset of the population. However, we must distinguish between
 - Target population
 - Sampled population

Ideally, these two overlap completely, but this does not always occur. Plus, we also need to consider the **sampling frame population**

I love the figure on page 5 of the book:

Within the **sampling frame** we have

- **Observation unit**
- **Sampling unit**

For example,

3. **Identifying potential sources of bias and ameliorating them:** In previous classes, like STAT 262 or STAT 380, we use the word **bias** to mean an estimator that systematically over- or under-estimates the true population characteristic. In sample surveys, we still use the same definition, but now biased estimators typically arise for two reasons:

- Population characteristic of interest (Estimation bias):
- Selection and/or measurement bias: Selection bias occurs when there is a mismatch between the sampled population and the target population. Measurement bias occurs when the survey observation tends to differ from the truth in a particular direction.

We'll talk about each of these in more detail.

Selection bias

- Convenience samples
- Judgment samples
- Self-selected samples
- Undercoverage
- Overcoverage
- Nonresponse

Measurement bias

Why can it occur?

- Unavoidable Reasons
- People!
- Survey mode: method used to distribute and gather responses

So we're concerned about response bias—what now? Careful question design is key. We could spend weeks on just questionnaire design, but will only do a very brief overview. There are other resources on campus that can help with this aspect of surveys (Bureau of Sociological Research).

We'll start with types of questions to avoid, and then move on to other considerations when writing questions.

Common types of biased questions:

- Leading questions

How to fix it:

- Assumptive questions

How to fix it:

- Double-barreled questions

How to fix it:

- Double negatives

How to fix it:

Other considerations:

- Pay careful attention to answer scale options

- Check formatting
- Avoid jargon, and be specific
- Open or closed questions?
- Question order matters

Probably most important, test questions ahead of time with the target population.

4. Choosing the sampling design/data collection approach

5. Choosing the analysis approach

6. Data analysis

7. **What's next?** The more information we have about a population, the easier it is to devise a sampling plan that will give accurate estimates. Any past survey can serve as a guide for future samples, as it will provide information about variability and cost.

The rest of the semester

2 An Introduction to Probability Samples

Before we can jump into sampling designs, we need to establish a few basic concepts and notation. The sampling designs we're going to discuss are **probability samples**.

💡 Definition: Probability sample

A **probability sample**

Throughout this semester we'll discuss three different ways to design a probability sample, as well as the properties of estimators arising from these designs. Probability samples allow us to avoid selection biases and help ensure the sample is representative of the population, as probability samples take selection choice away from the researcher, and put it in the hands of random selection. The three major designs we'll discuss are:

- Simple random sampling
- Stratified random sampling
- Cluster sampling

We'll also consider **systematic sampling**

To simplify things, we'll make the following assumptions:

Depending on how far we get, we may revisit relaxing some of these assumptions at the end of the semester. In all cases, our goal is to learn about some characteristic of the target population

and we'll use our sample data to estimate that characteristic. We'll also estimate standard errors, and use those to construct confidence intervals for the unknown population characteristic. We'll also discuss determining the necessary sample size.

2.1 Probability Sampling Definitions and Notation

Throughout this semester, we'll assume a finite population, or **universe** of N units:

Any sample we select from $\mathcal{U}$ will be a subset of $\mathcal{U}$, and we'll denote the specific sample chosen as $\mathcal{S}$. $\mathcal{S}$ contains n elements from $\mathcal{U}$.

Example: Suppose $\mathcal{U} = \{1, 2, 3, 4, 5\}$ and we consider samples of size $n = 3$. There are 10 possible samples that could be selected:

Remember that in **probability sampling**

Because each possible sample $\mathcal{S}$ has a known probability of being selected, each unit in the universe $\mathcal{U}$ also has a known probability of being in the selected sample.

Let y_i be

We consider y_i to be a *fixed quantity*.

2.2 Probability Samples: Inference Basics

We want to use the data from a sample to estimate some characteristic of the population. Common population characteristics of interest are

Example: Earlier we considered the universe $\mathcal{U} = \{1, 2, 3, 4, 5\}$. Suppose the values of y_i are

i	1	2	3	4	5
y_i	2	4	4	6	6

We also considered taking samples of size $n = 3$ from $\mathcal{U}$. There are 10 possible samples, and all are equally likely.

Just like in other classes like STAT 102, 380, 212 our inference methods are going to rely on the **sampling distribution** of a statistic.

 Definition: Sampling Distribution

The **sampling distribution**

Example, cont.:

From the sampling distribution, we can find properties of the estimator we're using.

- **Expected value:**

- Estimation bias:

- Variance:

- Mean squared error:

3 Simple Random Samples

Simple random sampling (SRS) is the most basic probability sampling design, and it is also used in combination with other designs like stratified and cluster sampling.

💡 Definition: Simple Random Sample

A **simple random sample (SRS)**

This definition implies

Specifically, the **inclusion probability** for any sampling unit is

Note that the converse is not true!

This is called an **equal probability** design, and the SRS is only one example of equal probability design. Some stratified and cluster sampling designs also have an equal inclusion probability for all sampling units.

3.1 SRS: Selecting the Sample

There are two types of selection procedures, depending on whether or not the sampling units are returned to the population after they have been drawn:

- SRSWR
- SRSWOR

Conceptually, an SRS (SRSWOR) is selected as follows:

-
-
-
-
- ⋮
-

This is useful to see why each unit has the same inclusion probability.

But, this is not an especially realistic way to pick a sample. In practice, an SRS (SRSWOR) is selected as follows:

- Create a sampling frame
- Determine a selection procedure that performs SRSWOR
- A sampling unit is included in the sample if its index corresponds to one of the n random numbers selected.

So what are some selection procedures that perform SRSWOR?

- Lottery
- [random.org](https://www.random.org)

The screenshot shows the Random.org website interface. At the top, the logo 'RANDOM.ORG' is displayed in large, bold, black letters. To the right of the logo is a search bar with the text 'Search RANDOM.ORG' and a 'Search' button. Below the logo, the text 'True Random Number Service' is visible. The main heading is 'Random Integer Set Generator'. Below this, a paragraph explains that the form allows generating random sets of integers from atmospheric noise. The 'Step 1: The Sets' section shows input fields for 'Generate 1 set(s) with 20 unique random integer(s) in each.' and 'Each integer should have a value between 1 and 100 (both inclusive; limits ±1,000,000,000)'. The 'Step 2: Display Options' section includes three checked checkboxes: 'Number the sets sequentially', 'Use commas to separate the set members', and 'Sort the members of each set in ascending order'. It also has three radio buttons for the order of sets: 'Print the sets in the order they were generated' (selected), 'Order the sets by the values that occur in them', and 'Print the sets in random order'. The 'Step 3: Go!' section has a message 'Be patient! It may take a little while to generate your sets...' and three buttons: 'Get Sets', 'Reset Form', and 'Switch to Advanced Mode'.

RANDOM.ORG Search RANDOM.ORG Search
True Random Number Service

Random Integer Set Generator

This form allows you to generate random sets of integers. The randomness comes from atmospheric noise, which for many purposes is better than the pseudo-random number algorithms typically used in computer programs.

Step 1: The Sets

Generate set(s) with unique random integer(s) in each.

Each integer should have a value between and (both inclusive; limits $\pm 1,000,000,000$).

The total number of integers must be no greater than 10,000.

Step 2: Display Options

Each set will be printed on a separate line. You can choose from the following extra options:

- ☒ Number the sets sequentially
- ☒ Use commas to separate the set members
- ☒ Sort the members of each set in ascending order

You can select the order in which the sets are printed:

- ☒ Print the sets in the order they were generated
- ☐ Order the sets by the values that occur in them (in this case, you should also consider sorting the members of each set)
- ☐ Print the sets in random order

Step 3: Go!

Be patient! It may take a little while to generate your sets...

Figure 3.1: Random.org settings to generate a SRSWOR

- Statistical software
 - SAS: `proc surveyselect method=srs;`
 - Base R: `sample(1:N, n, replace=FALSE)`
 - R packages: Need to install and load the package `sampling`, then `sample<-srswor(n,N)`. You can then use the `getdata` function to extract the sample from the population.

Example: The dataset `baseball.csv` in Canvas contains statistics on 797 baseball players from the rosters of all major league teams in November, 2004. We'll treat this as our population, and take an SRS of 150 players.

First, we'll read the data into R, print out the first few rows, and check the size of the population:

```
baseball<-read.csv("baseball.csv",header=TRUE)
head(baseball)
```

	team	leagueid	player	salary	pos	gplay	gstart	inning	putout	assist	error
1	ANA	AL	anderga0	6200000	CF	112	92	2375	211	5	2
2	ANA	AL	colonba0	11000000	P	3	34	625	8	30	3
3	ANA	AL	davanje0	375000	CF	108	27	743	75	1	0
4	ANA	AL	donnebr0	375000	P	5	0	126	2	2	0
5	ANA	AL	eckstda0	2150000	SS	142	136	3575	198	309	6
6	ANA	AL	erstada0	7750000	1B	125	124	3196	986	66	4
dplay pball gbat atbat run hit secbase thirdbase homerun rbi stolenb csteal											
1	1	-9	112	442	57	133	20	1	14	75	2
2	4	-9	3	3	0	0	0	0	0	0	0
3	1	-9	108	285	41	79	11	4	7	34	18
4	0	-9	5	0	0	0	0	0	0	0	0
5	75	-9	142	566	92	156	24	1	2	35	16
6	83	-9	125	495	79	146	29	1	7	69	16
walk strikeout intwalk hbp sacrhith sacrfly gidplay											
1	29		75	6	1		0	3		3	

2	0	1	0	0	0	0	0
3	46	54	2	0	1	5	2
4	0	0	0	0	0	0	0
5	42	49	1	13	14	2	11
6	37	74	1	4	3	4	9

```
N<-nrow(baseball)
N
```

```
[1] 797
```

Now we'll load the necessary package, set a seed and draw a sample of 150 players:

```
library(sampling)

set.seed(98263476)
n<-150
index<-srswor(n,N)
index[1:10]
```

```
[1] 0 0 0 0 0 0 0 1 0 1
```

So the `srswor` function only gives us a 0/1 for each element in the population. To get the sample, there are a couple of options:

```
baseballsrs<-getdata(baseball,index)
head(baseballsrs)
```

	ID_unit	team	leagueid	player	salary	pos	gplay	gstart	inning	putout	assist	
8	8	ANA	AL	figgich0	320000	3B	148	80	2116	57	129	
10	10	ANA	AL	greggke0	301500	P	5	0	263	2	5	
21	21	ANA	AL	rodriifr0	375000	P	3	0	252	5	9	
31	31	ARI	NL	cintral0	335000	SS	154	125	3297	141	383	
33	33	ARI	NL	daiglca0	300000	P	11	10	147	5	6	
50	50	ARI	NL	sexsori0	8725000	1B	23	23	613	199	27	
	error	dplay	pball	gbat	atbat	run	hit	seabase	thirdbase	homerun	rbi	stolenb
8	11	9	-9	148	577	83	171	22	17	5	60	34
10	0	1	-9	5	0	0	0	0	0	0	0	0
21	0	0	-9	3	0	0	0	0	0	0	0	0
31	15	61	-9	154	564	56	148	31	7	4	49	3
33	0	0	-9	11	17	2	2	2	0	0	0	0
50	1	19	-9	23	90	20	21	4	0	9	23	0

	csteal	walk	strikeout	intwalk	hbp	sacrhit	sacrfly	gidplay
8	13	49	94	0	0	10	2	6
10	0	0	0	0	0	0	0	0
21	0	0	0	0	0	0	0	0
31	3	31	59	2	2	12	4	11
33	0	0	7	0	0	1	0	1
50	0	14	21	0	0	0	0	2

```
baseballsrs2<-baseball[(1:N)[index==1],]
head(baseballsrs2)
```

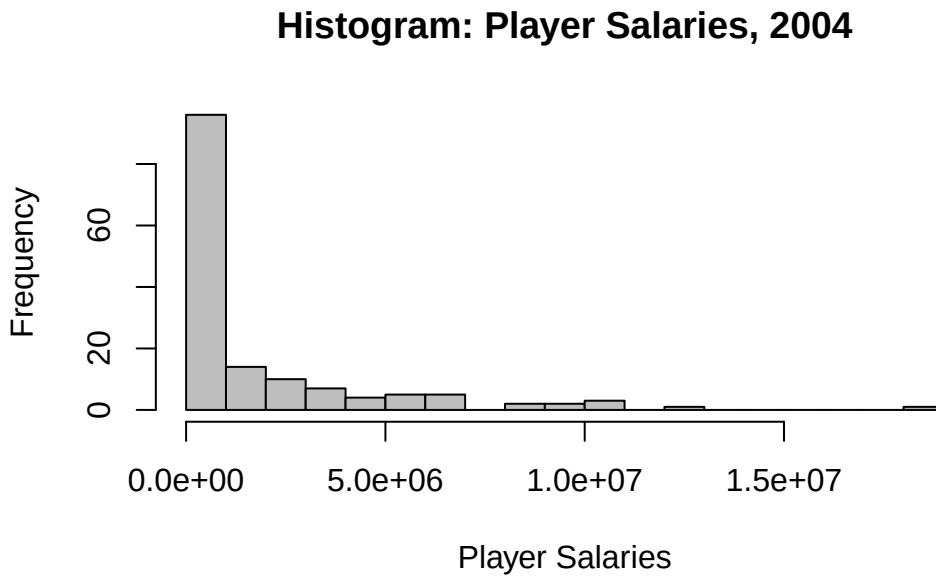
	team	leagueid	player	salary	pos	gplay	gstart	inning	putout	assist	error
8	ANA	AL	figgich0	320000	3B	148	80	2116	57	129	11
10	ANA	AL	greggke0	301500	P	5	0	263	2	5	0
21	ANA	AL	rodriifr0	375000	P	3	0	252	5	9	0
31	ARI	NL	cintral0	335000	SS	154	125	3297	141	383	15
33	ARI	NL	daiglca0	300000	P	11	10	147	5	6	0
50	ARI	NL	sexsori0	8725000	1B	23	23	613	199	27	1

	dplay	pball	gbat	atbat	run	hit	secbase	thirdbase	homerun	rbi	stolenb	csteal
8	9	-9	148	577	83	171	22	17	5	60	34	13
10	1	-9	5	0	0	0	0	0	0	0	0	0
21	0	-9	3	0	0	0	0	0	0	0	0	0
31	61	-9	154	564	56	148	31	7	4	49	3	3
33	0	-9	11	17	2	2	2	0	0	0	0	0
50	19	-9	23	90	20	21	4	0	9	23	0	0

	walk	strikeout	intwalk	hbp	sacrhit	sacrfly	gidplay
8	49	94	0	0	10	2	6
10	0	0	0	0	0	0	0
21	0	0	0	0	0	0	0
31	31	59	2	2	12	4	11
33	0	7	0	0	1	0	1
50	14	21	0	0	0	0	2

Let's plot the salaries from our sampled players

```
hist(baseballsrs$salary,breaks=20,col="gray",xlab="Player Salaries",  
      main="Histogram: Player Salaries, 2004")
```



3.2 Estimation from a SRS

Once we've selected our sample, we can use it to estimate characteristics of the population, along with their standard errors.

3.2.1 Estimating a population mean

As a reminder, the population mean is

A natural estimator of $\bar{y}_U$ using a SRS of sample size n is the **sample mean**:

Example: In our sample of $n = 150$ baseball players, the mean salary is

```
mean.salary<-mean(baseballsrs$salary)
mean.salary
```

```
[1] 1993886
```

Because we have access to the entire population, we can check to see how our sample mean compares to the population mean.

```
mean.pop<-mean(baseball$salary)
mean.pop
```

```
[1] 2497669
```

What happens if we take a different sample? Let's try a few. What do we need to change in R?

It turns out that with simple random sampling, the sample mean is an unbiased estimator of the population mean. That is,

As noted in Chapter 2, the finite population variance is

Under simple random sampling, the sample variance s^2

is an unbiased estimator of σ^2 . However, if we want to use $\bar{y}$ to draw conclusions about the population, we need the variance of $\bar{y}$. It turns out that under simple random sampling,

(Proof: see section 2.9)

But, we generally don't know σ^2 , so we'll use an unbiased estimator of $\text{Var}(\bar{y})$:

The square root of the variance of the estimator is its standard error (fun fact: standard errors are not usually unbiased).

This quantity

is called the **finite population correction factor**, or **fpc** for short.

Keep in mind that we've sampled $\frac{n}{N}$ of the population. This is called the **sampling fraction**. Why does the sampling fraction matter?

Example: For the baseball SRS

We know:

With the fpc:

```
var.salary<-var(baseballsrs$salary)
var.salary
```

```
[1] 8.804137e+12
```

```
se.mean<-sqrt((var.salary/n)*(1-(n/N)))
se.mean
```

```
[1] 218283.4
```

Without the fpc:

```
se.nofpc<-sqrt(var.salary/n)
se.nofpc
```

```
[1] 242269
```

What if we had sampled fewer players? Let's try 30 instead of 150. Now:

```
set.seed(98263476)
sampsiz<-30
indexsmall<-srswor(sampsiz,N)
srssmalln<-getdata(baseball,indexsmall)
meansmall<-mean(srssmalln$salary)
meansmall
```

```
[1] 1904472
```



```
var.small<-var(srssmalln$salary)
var.small
```

```
[1] 4.76297e+12
```

```
se.small<-sqrt((var.salary/sampsize)*(1-(sampsize/N)))
se.small
```

```
[1] 531436.4
```

```
nofpc.small<-sqrt(var.salary/sampsize)
nofpc.small
```

```
[1] 541729.8
```

We may also be interested in a measure of relative variability (this will be most helpful in the future for selecting sample size). The **coefficient of variation** (CV) of $\bar{y}$ is

The CV does not depend on the units of measurement. We can estimate the CV using

Example: For the baseball SRS, the estimated CV is

```
CV<-se.mean/mean.salary
CV
```

```
[1] 0.1094764
```

The salary data are quite skewed, so we should also report the median:

```
med.salary<-median(baseballsrs$salary)
med.salary
```

```
[1] 525000
```

3.2.2 Estimating a population total

As a reminder, the population total is

We can also express the population total as a function of the population mean

It's then natural to estimate t using

It's easy to see that

The CV doesn't change because the N s cancel!

Example: Let's estimate the total number of runs across the 2004 season, based on our SRS of $n = 150$ players. First, let's start with the mean runs

```
mean.runs<-mean(baseballsrs$run)
mean.runs
```

```
[1] 24.39333
```

```
var.runs<-var(baseballsrs$run)
var.runs
```

```
[1] 998.2268
```

```
se.meanrun<-sqrt((var.runs/n)*(1-(n/N)))
se.meanrun
```

```
[1] 2.324299
```

We can now use these to find an estimate of the total number of runs, along with its standard error:

```
tot.runs<-N*mean.runs
tot.runs
```

```
[1] 19441.49
```

```
se.totrun<-N*se.meanrun
se.totrun
```

```
[1] 1852.466
```

Again, since we have the whole population we can calculate the true total number of runs:

```
pop.totrun<-N*mean(baseball$run)
pop.totrun
```

```
[1] 20515
```

3.2.3 Estimating a population proportion

A proportion is just a special case of a mean! Let p be the proportion of the population that has a characteristic, and define y_i as

Then,

and $\hat{p}$ is an unbiased estimator for p .

Now let's consider the variance.

If we plug $\hat{p}$ in for p :

Example: Let's estimate the population proportion of players who had at least one home run. First, we'll need to do some fiddling with the data set.

```
baseballsrs$hr<-rep(0,n)
baseballsrs$hr[baseballsrs$homerun>0]<-1
head(baseballsrs)
```

	ID_unit	team	leagueid	player	salary	pos	gplay	gstart	inning	putout	assist
8	8	ANA	AL	figgich0	320000	3B	148	80	2116	57	129
10	10	ANA	AL	greggke0	301500	P	5	0	263	2	5
21	21	ANA	AL	rodrifr0	375000	P	3	0	252	5	9
31	31	ARI	NL	cintral0	335000	SS	154	125	3297	141	383
33	33	ARI	NL	daiglca0	300000	P	11	10	147	5	6
50	50	ARI	NL	sexsori0	8725000	1B	23	23	613	199	27

	error	dplay	pball	gbat	atbat	run	hit	secbase	thirdbase	homerun	rbi	stolenb
8	11	9	-9	148	577	83	171	22	17	5	60	34
10	0	1	-9	5	0	0	0	0	0	0	0	0
21	0	0	-9	3	0	0	0	0	0	0	0	0
31	15	61	-9	154	564	56	148	31	7	4	49	3
33	0	0	-9	11	17	2	2	2	0	0	0	0
50	1	19	-9	23	90	20	21	4	0	9	23	0

	csteal	walk	strikeout	intwalk	hbp	sacrhit	sacrflly	gidplay	hr
8	13	49	94	0	0	10	2	6	1
10	0	0	0	0	0	0	0	0	0
21	0	0	0	0	0	0	0	0	0
31	3	31	59	2	2	12	4	11	1
33	0	0	7	0	0	1	0	1	0
50	0	14	21	0	0	0	0	2	1

Now we can use our new variable to estimate the proportion of all players that had at least one homerun:

```
prop.hr<-mean(baseballsrs$hr)
prop.hr
```

```
[1] 0.5066667
```

and find its standard error

```
fpc<-1-(n/N)
fpc
```

```
[1] 0.8117942
```

```
var.prophr<-(fpc*prop.hr*(1-prop.hr))/(n-1)
var.prophr
```

```
[1] 0.001361829
```

```
se.prophr<-sqrt(var.prophr)
se.prophr
```

```
[1] 0.03690296
```

3.3 Sampling Weights

Earlier, we defined the **inclusion probability** (π_i) for each sampling unit. This is the probability that unit i is included in the sample. In probability samples, the inclusion probabilities are used to calculate $\bar{y}$ and $\hat{t}$.

To see how they are used, we need a new concept—the **sampling weight**.

 Definition: Sampling weight

The **sampling weight**

For example, with the baseball data

The sampling weight is also sometimes called the **design weight**.

For simple random samples, all sampling units have the same inclusion probability:

This means that all sampling units also have the same sampling weight:

Because all units have the same sampling weight, an SRS is a **self-weighting sample**. However, just because a sample is self-weighting does not mean it's an SRS. It's possible to have a sample design where every unit has the same inclusion probability, but not all possible samples have the same chance to be selected.

As mentioned above, we can use the sampling weights w_i to calculate $\bar{y}$ and $\hat{t}$ (and, thus, $\hat{p}$).

For an SRS, $w_i = N/n$, so

We didn't need to use them in baseball example to get $\bar{y}$, $\hat{t}$, and $\hat{p}$. But, survey software packages in R and SAS are written to estimate the parameters for any probability sampling design so they require the data set include sampling weights.

Example: Let's add the sampling weights to the baseball data.

```
baseballsrs$sampwt<-rep(N/n,n)
head(baseballsrs)
```

	ID_unit	team	leagueid	player	salary	pos	gplay	gstart	inning	putout	assist
8	8	ANA	AL	figgich0	320000	3B	148	80	2116	57	129

10	10	ANA	AL	greggke0	301500	P	5	0	263	2	5	
21	21	ANA	AL	rodrifr0	375000	P	3	0	252	5	9	
31	31	ARI	NL	cintral0	335000	SS	154	125	3297	141	383	
33	33	ARI	NL	daiglca0	300000	P	11	10	147	5	6	
50	50	ARI	NL	sexsori0	8725000	1B	23	23	613	199	27	
error dplay pball gbat atbat run hit secbase thirdbase homerun rbi stolenb												
8	11	9	-9	148	577	83	171	22	17	5	60	34
10	0	1	-9	5	0	0	0	0	0	0	0	0
21	0	0	-9	3	0	0	0	0	0	0	0	0
31	15	61	-9	154	564	56	148	31	7	4	49	3
33	0	0	-9	11	17	2	2	2	0	0	0	0
50	1	19	-9	23	90	20	21	4	0	9	23	0
csteal walk strikeout intwalk hbp sacrhith sacrfly gidplay hr sampwt												
8	13	49		94	0	0	10	2	6	1	5.313333	
10	0	0		0	0	0	0	0	0	0	5.313333	
21	0	0		0	0	0	0	0	0	0	5.313333	
31	3	31		59	2	2	12	4	11	1	5.313333	
33	0	0		7	0	0	1	0	1	0	5.313333	
50	0	14		21	0	0	0	0	2	1	5.313333	

The sampling weight is $N/n = 797/150 = 5.31333$, indicating that player in our SRS represents 5.31333 players in the population of all 2004 MLB players.

We can then use the sampling weights to calculate the same estimates we did earlier, using a built in function in the **survey** package in R. We first have to define the survey design, using the **svydesign** function:

```
library(survey)
baseballdesign<-svydesign(id=~1, weights=~sampwt, fpc=rep(N,n), data=baseballsrs)
```

The **svydesign** function has several inputs:

- id
- weights
- fpc
- data

There are other inputs that we don't need right now, but we'll add them as we encounter more complicated sampling designs.

Let's look the information included in `baseballdesign`:

```
baseballdesign
```

Independent Sampling design

```
svydesign(id = ~1, weights = ~sampwt, fpc = rep(N, n), data = baseballsrs)
```

R tells us this is an Independent Sampling Design, which means there are no clusters or strata.

Now that we've set up the sampling design in R, we can use it calculate the same estimates we did before with a lot less code. We can use the functions `svymean` and `svytotal` to get $\bar{y}$ and $\hat{t}$, respectively, along with their standard errors. The generic code is:

```
ybar<-svymean(~var_interest, design)
```

```
that<-svytotal(~var_interest, design)
```

Example: For estimating average salary and total runs in 2004 we'd use:

```
salmean<-svymean(~salary,baseballdesign)
salmean
```

	mean	SE
salary	1993886	218283

```
runtotal<-svytotal(~run,baseballdesign)
runtotal
```

	total	SE
run	19442	1852.5

The same values we got before. Since $\hat{p}$ is really a mean, we can also use `svymean` to get $\hat{p}$ and its SE:

```
hrprop<-svymean(~hr,baseballdesign)
hrprop
```

	mean	SE
hr	0.50667	0.0369

Again, the same values we got before.

3.4 Confidence Intervals

Once we select the random sample and use the resulting data to construct estimates of a population value (mean, total, proportion), the next step is typically to make some statement about the accuracy of the estimate. This is most often accomplished by constructing a **confidence interval** for the true population value.

The confidence intervals we'll construct have the same interpretation as those we constructed in classes like STAT 218, 318, 102, 212, 380.

💡 Definition: Confidence interval

A **confidence interval**

The expected proportion of intervals containing the true population value is the **coverage probability**. So, for example, an interval with a coverage probability of 0.95 means

But, there are differences. In probability sampling with a finite population, only a finite number of possible samples exist, and we know the probability with which each sample will be chosen. This means we *could* generate all possible samples, and all possible intervals. From these, we could then observe how many capture the population value, and get the *exact* coverage probability.

In real life, we can't do this

But, the confidence intervals we used in STAT 218/318/102/212/380 (which assumed an infinite population) relied on asymptotic theory—what happens as $n \rightarrow \infty$. For example, we know and love the Central Limit Theorem:

But we've only got a finite population! How can we use a result about $n \rightarrow \infty$ when we've got a finite population?

Happily, there is a version of the Central Limit Theorem for SRSWOR, due to Hájek (1960). If some conditions that are *way* too technical for us to talk about are true and if

then

This means we could use this confidence interval for the population mean:

Again, we don't know σ ! We could substitute in s and use the standard error. And, in practice, we typically substitute $t_{\alpha/2, df=n-1}$ for $z_{\alpha/2}$. In small samples, it's more conservative (wider confidence intervals) and with large samples $t_{\alpha/2, n-1} \approx z_{\alpha/2}$ anyway. So, we'll use the interval

to calculate a $(1 - \alpha) \times 100\%$ confidence interval for the from population mean from an SRS.

In general,

We have the same question come up that comes with the regular CLT: how large is large enough for n (and N ?) It depends

Some rules of thumb:

- If the population is approximately normal,
-

Sugden et al. (2000) recommended the minimum sample size

Another way to check whether the distribution is too skewed to use the CLT is bootstrapping!

Example: Back to the baseball data. Let's get a 95% confidence interval for the mean salary of players in 2004. We saw already that based on our SRS of $n = 150$, $\bar{y} = 1993886$ and $SE(\bar{y}) = 218283.4$. We can use the `qt(alpha/2,df)` function to get the t multiplier in the confidence interval.

```
tcrit<-qt(0.025,n-1,lower.tail=FALSE)
lower<-mean.salary-tcrit*se.mean
upper<-mean.salary+tcrit*se.mean
lower
```

```
[1] 1562555
```

```
upper
```

```
[1] 2425217
```

Should we have even used the t -interval for the salary data? It's pretty skewed. We can find the skewness to check.

```
library(moments)
salskew<-skewness(baseballsr$salary)
salskew
```

```
[1] 2.538835
```

And using the Sudgen et al. (2000) recommendation, we need at least

```
nmin<-28 + 25*(salskew)**2
nmin
```

```
[1] 189.142
```

What about a 95% confidence interval for the total number of runs in 2004? We already found $\hat{t} = 19441.49$ and $SE(\hat{t}) = 1852.4661651$.

```
lowerrun<-tot.runs-tcrit*se.totrun  
upperrun<-tot.runs+tcrit*se.totrun  
lowerrun
```

```
[1] 15780.99
```

```
upperrun
```

```
[1] 23101.98
```

Let's try a 95% confidence interval for the proportion players with at least one home run. We know $\hat{p} = 0.5066667$ and $SE(\hat{p}) = 0.036903$.

```
lowerhr<-prop.hr-tcrit*se.prophr  
upperhr<-prop.hr+tcrit*se.prophr  
lowerhr
```

```
[1] 0.4337459
```

```
upperhr
```

```
[1] 0.5795874
```

It turns out the `survey` package has confidence intervals built in to the `svymean` and `svytotal` functions. Once we have created an object from either function, we can pass that object into the `confint` function. Here's the syntax:

```
confint(object,df= , level= )
```

So we could use

```
salmean<-svymean(~salary, baseballdesign)
salmean
```

```
      mean      SE
salary 1993886 218283
```

```
confint(salmean,df=n-1, level=0.95)
```

```
      2.5 %  97.5 %
salary 1562555 2425217
```

For more practice:

- draw a new sample of size $n = 150$ (with no seed or pick your own seed, so everyone gets a different sample) from the baseball data set
- use `svymean` to a 90% confidence interval for the mean number of errors committed by a player in 2004

3.5 Sample Size

Often the first question that comes up when designing a survey is how many observations are needed. This is often a pretty complicated question, and involves balancing the objectives of the study with the available budget. You also must consider that most studies have multiple characteristics being measured on each observation unit.

If the researcher can narrow down the most important variables to focus on one or two, there are four basic steps:

1. Decide how much precision you need (basically, how much variability are you willing to live with in the estimates)
2. Find an equation involving the sample size n and the precision (i.e., the margin of error)
3. Estimate anything you need to and solve for n (usually an estimate of the expected variance)
4. Go back to square one when the n turns out to be enormous.

We'll go through these steps one by one.

1. Decide how much precision you need

2. Find an equation involving n and the precision

Let's start by assuming the fpc is negligible (i.e., we have an infinite population)

Now, let's assume $\frac{n_0}{N} < 1$ (why is this a reasonable assumption?)

So, the sample size we need is

We could also consider finding the sample size to achieve a specified relative precision, call it r . The relative precision measures the distance between the estimate and the true value, scaled by the true value.

Sometimes we can calculate the required n at this stage!

Example: Suppose we want to estimate the proportion of 2004 baseball players that had at least one error, and we want to be able to calculate a 95% confidence interval and have a margin of error of 0.04.

3. Estimate anything you need to and solve for n

This usually involves estimating (guessing) the expected variance. Why didn't need to do this in the baseball example above?

How can we obtain the expected variance?

Another option is to use the CV for a sample of size 1: $\sigma/\bar{y}_u$

If there are no other options, guess. An educated guess may use the suspected distribution of the data.

4: Go back to square one when the n turns out to be enormous

Example: A botanical researchers wishes to design a survey to estimate the number of birch trees in a study area. The study area has been divided into 1000 plots. From previous experience, the variance in the number of trees per plot is about $\sigma^2 \approx 45$. What sample size should be used to estimate the total number of trees in the study area to within 500 trees of the true value with 95% confidence?

To within 1000 trees?

To within 2000 trees?

Does the finite population correction factor matter here? How do you know?

3.6 Systematic Sampling

Suppose we have a population with N units, and the units are numbered from 1 to N in some order. To select a sample of n units, we:

For example,

Note that

This type of sampling design is called a **systematic sample**

Why isn't this a SRS?

When will a systematic sample behave like an SRS?

When will a systematic sample not behave like an SRS?

There are a couple of advantages over the SRS:

3.7 Considerations for Choosing an SRS Design

When Shouldn't You Use an SRS?

- You really should be carrying out a designed experiment
- You don't have a sampling frame or it's too expensive
- You've got additional information

Why Should You Use an SRS?

- You don't have extra information
- Explainability is super important
- Easier computationally